PROGRAMA DE TRAINEES EM DADOS E INTELIGÊNCIA ARTIFICIAL

AI Talent Academy

Semana 3 · Aula 5 · SQL para Análise de Dados

White Cube
A Think Group Company

Segunda-feira, 19h00 às 22h00 - Online

whitecube.com.br



ENQUANTO TODO MUNDO ENTRA

Aula de hoje é sobre Structured Query Language (SQL)

Aula 5 · Semana 3 · começamos 19h10

- **Áudio e câmera:** microfone no mudo, câmera fica a seu critério
- **Dúvidas:** escreva no chat a qualquer momento, temos moderação acompanhando
- **Gravação:** a aula é gravada e o link chega em até 24h
- **Material:** slides e resumo vão para o Discord depois da aula
- **Aqueça o teclado e escreva no chat**: você já usa filtro ou tabela dinâmica numa planilha?
- **Além disso**: já utilizou SQL alguma vez?

ONDE ESTAMOS

A jornada das 8 semanas

Semana 3 de 8. A primeira vez que “tocaremos” de fato nos dados.

Semana	Tema	Foco
1	Fundamentos, nivelamento e panorama	Todo mundo junto. Base comum
2	IA Generativa	LLM por dentro, prompt e limites
3 (hoje)	Análise de Dados com SQL	Consultar e extrair. A base de toda análise
4	Análise de Dados com Python	Manipular e visualizar. Storytelling com dados
5	Engenharia de Dados	Construir os pipelines. Dado limpo, no lugar certo
6	Machine Learning	Prever com dados. Do baseline ao modelo
7	Agentes de IA	Da resposta à ação. IA que executa tarefas
8	Apresentação final	Demo Day. Solução completa, do problema ao resultado

O PLANO DE HOJE

Quatro blocos, sempre voltando ao mesmo caso

Todo exemplo parte do mesmo caso: fraude em cartão, da Aula 2.

Anatomia de uma consulta

SELECT, FROM, WHERE, ORDER BY e LIMIT.

35 minutos

Filtros e condições

Refinar o que a consulta traz de volta.

25 minutos

Agregações

Resumir muitas linhas numa resposta só.

35 minutos

Juntando tabelas

JOIN, e por que os dados vivem separados.

30 minutos

White Cube
A Think Group Company

01

Por que SQL?

A ferramenta mais antiga da área, ainda uma das mais usadas todos os dias

19h10 às 19h25 · 15 minutos

SQL

DA PESQUISA À LINGUAGEM UNIVERSAL DOS DADOS

SQL tem mais de 50 anos. Por que ainda usamos?

- **1970: O modelo relacional**

Edgar F. Codd, pesquisador da IBM, propõe o modelo relacional: representar dados em relações e permitir que usuários trabalhem com eles sem precisar conhecer todos os detalhes de armazenamento.

- **1974: Nasce o SEQUEL**

Donald Chamberlin e Raymond Boyce desenvolvem na IBM uma linguagem para consultar bancos relacionais. Inicialmente chamada SEQUEL, ela daria origem ao SQL.

- **Anos 1980: SQL vira padrão**

A linguagem se espalha pelos bancos comerciais e é padronizada por ANSI e ISO, consolidando uma forma comum de conversar com bancos relacionais.

- **Hoje: A tecnologia mudou. A ideia permaneceu.**

PostgreSQL, SQL Server, Oracle, MySQL, BigQuery, Snowflake, Databricks e inúmeras outras plataformas continuam utilizando SQL ou linguagens fortemente baseadas nele.



Donald Chamberlin



Raymond Boyce

White Cube
A Think Group Company

02

Anatomia de uma consulta

SELECT, FROM, WHERE, ORDER BY e LIMIT

19h25 às 20h00 · 35 minutos

O CASO DE HOJE

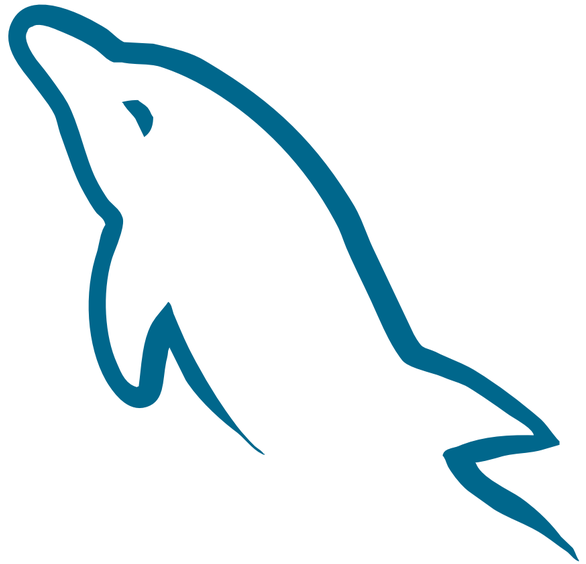
Três tabelas, o mesmo caso da Aula 2

O mesmo caso de fraude em cartão da Aula 2. Toda consulta de hoje parte daqui.

Tabela	O que guarda	Colunas principais
transacoes	Cada compra no cartão	id_transacao, id_cliente, id_lojista, valor, data_hora, canal, status
clientes	Quem é o dono do cartão	id_cliente, nome, cidade, data_cadastro
lojistas	Onde a compra aconteceu	id_lojista, nome, categoria, cidade

SQL não é “programação” no sentido comum

- **SQL é uma linguagem declarativa:** você descreve o que quer, não o passo a passo de como buscar
- **O banco decide o caminho mais rápido:** por trás da consulta existe um otimizador escolhendo a rota
- **A base de tudo são cinco palavras:** SELECT, FROM, WHERE, ORDER BY e LIMIT resolvem boa parte das perguntas do dia a dia



SELECT e FROM: de onde vem o dado

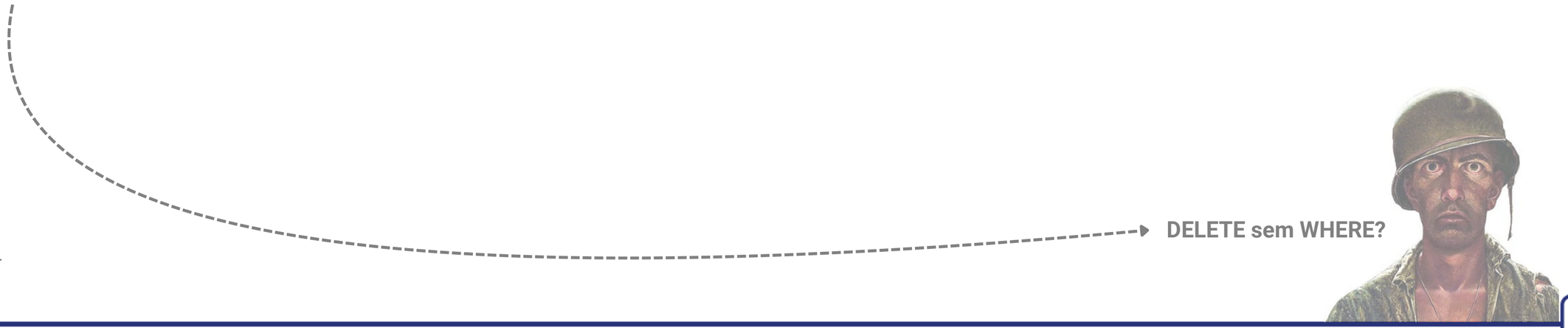
```
SELECT nome, cidade
FROM clientes
```

- **SELECT escolhe as colunas:** só o que você pede aparece no resultado
- **FROM escolhe a tabela:** de onde essas colunas são lidas
- **A ordem das colunas no resultado segue o SELECT:** não a ordem da tabela original
- **`SELECT *` traz todas as colunas:** prático para explorar, mas evite deixar assim nos relatórios finais

WHERE: filtrando antes de trazer o resultado

```
SELECT nome, cidade
FROM clientes
WHERE cidade = 'Curitiba'
```

- **WHERE decide quais linhas entram:** aqui, só clientes de Curitiba aparecem no resultado
- **Texto vai entre aspas simples:** número não precisa de aspas
- **Sem ‘WHERE’, a consulta traz a tabela inteira:** numa tabela grande, isso pode travar o banco



ORDER BY e LIMIT: ordenando e cortando

```
SELECT nome, valor
FROM transacoes
ORDER BY valor DESC
LIMIT 10
```

- **ORDER BY organiza o resultado:** ASC do menor para o maior, DESC do maior para o menor
- **LIMIT corta o tamanho do resultado:** aqui, só as 10 transações de maior valor aparecem
- **Juntos, respondem quais são os maiores:** um dos pedidos mais comuns em qualquer análise

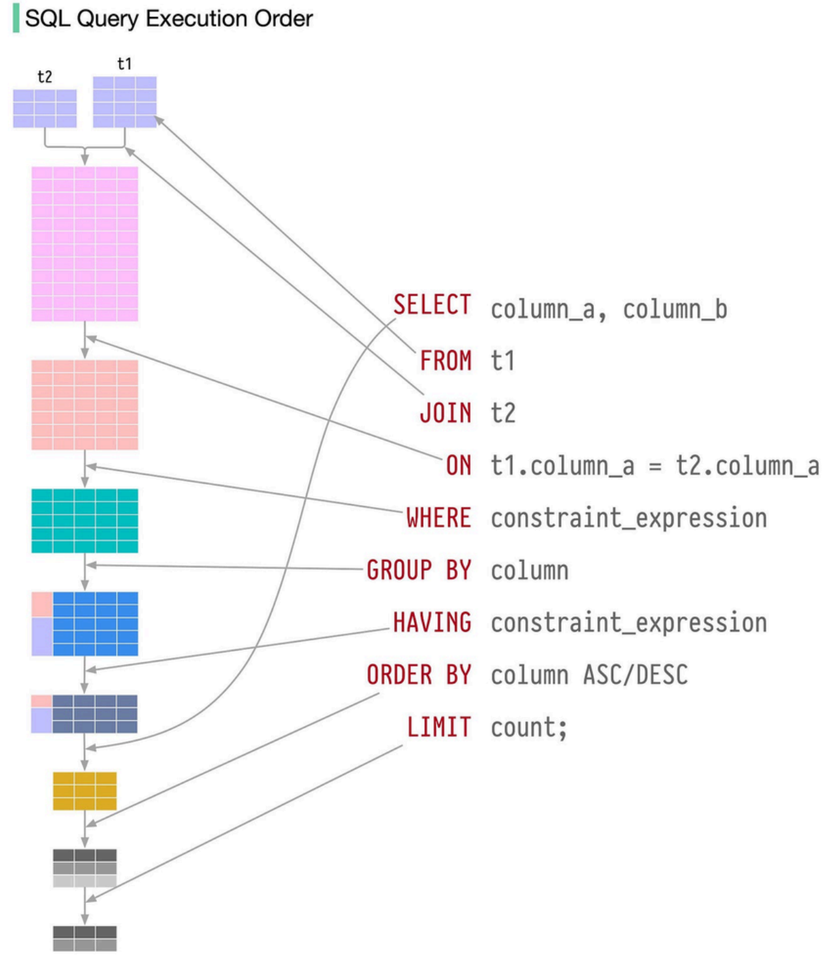
White Cube
A Think Group Company

A ORDEM REAL

A ordem que você escreve não é a ordem que o banco executa

Você escreve SELECT primeiro, mas o banco lê FROM primeiro.

1. **FROM:** o banco decide primeiro de qual tabela vai ler
2. **WHERE:** filtra as linhas, antes de escolher qualquer coluna
3. **GROUP BY:** agrupa as linhas que sobraram, quando existe agrupamento
4. **HAVING:** filtra os grupos, depois de agrupar
5. **SELECT:** só agora escolhe as colunas do resultado
6. **ORDER BY:** organiza o que sobrou
7. **LIMIT:** corta o tamanho, por último



INTERAÇÃO

Qual filtro resolve isso

Você quer as transações entre R$100 e R$300, feitas em São Paulo ou Curitiba. Qual cláusula resolve isso de um jeito só?

A: WHERE valor > 100 AND cidade = 'São Paulo'
B: WHERE valor BETWEEN 100 AND 300 AND cidade IN ('São Paulo', 'Curitiba')
C: WHERE valor = 200 OR cidade = 'São Paulo'
D: WHERE valor = BETWEEN 100 AND 300 AND cidade = ('São Paulo', 'Curitiba')
E: WHERE valor = 200 OR 300 OR cidade = 'São Paulo' OR ‘Curitiba’

Responda na enquete: 60 segundos

White Cube
A Think Group Company

03

Filtrando e refinando

Operadores, combinações e o cuidado com valor vazio

20h00 às 20h25 · 25 minutos

Comparando e testando faixas de valor

```
SELECT id_transacao, valor
FROM transacoes
WHERE valor > 500
    AND valor <= 2000
```

- **Operadores de comparação:** maior que, menor que, maior ou igual, menor ou igual, diferente
- **AND exige que as duas condições sejam verdadeiras:** só entra quem passa nas duas ao mesmo tempo
- **BETWEEN faz a mesma faixa numa linha só:** WHERE valor BETWEEN 500 AND 2000

OR e IN: combinando opções possíveis

```
SELECT nome, categoria
FROM lojistas
WHERE categoria = 'Eletrônicos'
   OR categoria = 'Joalheria'
```

- **OR exige que pelo menos uma condição seja verdadeira:** entra quem passa em qualquer uma das duas
- **IN substitui uma fila de OR na mesma coluna:** WHERE categoria IN ('Eletrônicos', 'Joalheria')
- **Cuidado ao misturar AND e OR:** sem parênteses, o resultado pode não ser o que você imaginou

Texto e valor vazio

```
SELECT nome
FROM clientes
WHERE nome LIKE 'Ana%'
   AND cidade IS NOT NULL
```

- **LIKE busca por padrão de texto:** o símbolo % substitui qualquer sequência de caracteres
- **Valor vazio no banco não é zero nem texto vazio:** é NULL, um estado próprio, sem valor definido
- **Por isso NULL não se compara com igual:** usa IS NULL ou IS NOT NULL, nunca igual a NULL

White Cube
A Think Group Company

INTERVALO

15 minutos

Voltamos às 20h45 em ponto.



Enquanto isso: entre no Discord do AI Talent Academy (link no chat)
Deixe sua dúvida no chat. Vamos responder na volta.

White Cube
A Think Group Company

04

Agregações

Resumindo muitas linhas numa resposta só

20h40 às 21h15 · 35 minutos

A pergunta muda de forma

- **Até aqui, cada linha do resultado é uma transação:** 'Quais são as compras acima de R$500 em São Paulo?'
- **Agora a pergunta muda de forma:** 'Quanto São Paulo faturou no total?' resume tudo num número só
- **Cinco funções resolvem boa parte dos casos:** COUNT, SUM, AVG, MIN e MAX

AGREGAÇÕES

Uma resposta só, agregada a partir de muitas linhas

```
SELECT COUNT(*) AS total_transacoes,
       SUM(valor) AS valor_total,
       AVG(valor) AS valor_medio
FROM transacoes
```

- **COUNT conta linhas:** aqui, quantas transações existem no total
- **SUM soma um valor numérico:** o total movimentado em todas as transações
- **AVG calcula a média:** MIN e MAX trazem o menor e o maior valor da coluna
- **AS dá um nome ao resultado da coluna:** sem ele, a coluna sai com um nome pouco legível

GROUP BY: agrupando antes de agregar

```
SELECT id_lojista, COUNT(*) AS total,
       SUM(valor) AS valor_total
FROM transacoes
GROUP BY id_lojista
```

- **GROUP BY separa as linhas em grupos:** aqui, um grupo para cada lojista
- **A função agregada roda dentro de cada grupo:** COUNT e SUM calculam por lojista, não pela tabela inteira
- **Toda coluna do SELECT precisa estar no GROUP BY, ou dentro de uma função:** senão o banco não sabe qual linha do grupo mostrar

HAVING: filtrando depois de agrupar

```
SELECT id_lojista, SUM(valor) AS valor_total
FROM transacoes
GROUP BY id_lojista
HAVING SUM(valor) > 10000
```

- **WHERE filtra antes de agrupar, HAVING filtra depois:** são dois momentos diferentes da mesma consulta
- **HAVING trabalha com o resultado da agregação:** aqui, só lojistas que somaram mais de R$10.000
- **Usar SUM dentro do WHERE dá erro:** nesse ponto da execução, a soma ainda não existe

White Cube
A Think Group Company

05

Juntando tabelas

JOIN, chave primária e chave estrangeira

21h15 às 21h45 · 30 minutos

Cada tabela guarda um assunto só

- **Guardar tudo numa tabela só parece mais simples:** mas repete nome e cidade do cliente em toda transação dele
- **Separar em tabelas evita essa repetição:** o nome do cliente mora uma vez só, na tabela clientes
- **A ligação entre tabelas é feita por uma chave:** um identificador que aparece nas duas tabelas

Chave primária e chave estrangeira

É esse par que o JOIN usa para juntar as tabelas.

Termo	O que é	Exemplo no nosso caso
Chave primária	identifica cada linha de forma única, dentro da própria tabela	id_cliente, dentro de clientes
Chave estrangeira	aponta para a chave primária de outra tabela	id_cliente, dentro de transacoes

JOIN: juntando transação com o nome do cliente

```
SELECT t.id_transacao, c.nome, t.valor
FROM transacoes t
JOIN clientes c
   ON t.id_cliente = c.id_cliente
```

- **JOIN junta linhas de duas tabelas:** usando a chave que elas têm em comum
- **ON diz qual coluna de cada lado combina:** aqui, o id_cliente de um lado com o id_cliente do outro
- **“t” e “c” são apelidos das tabelas:** encurtam o nome e evitam ambiguidade quando a coluna se repete
- **Sem JOIN, esse cruzamento teria que ser feito manualmente:** fora do banco, numa planilha

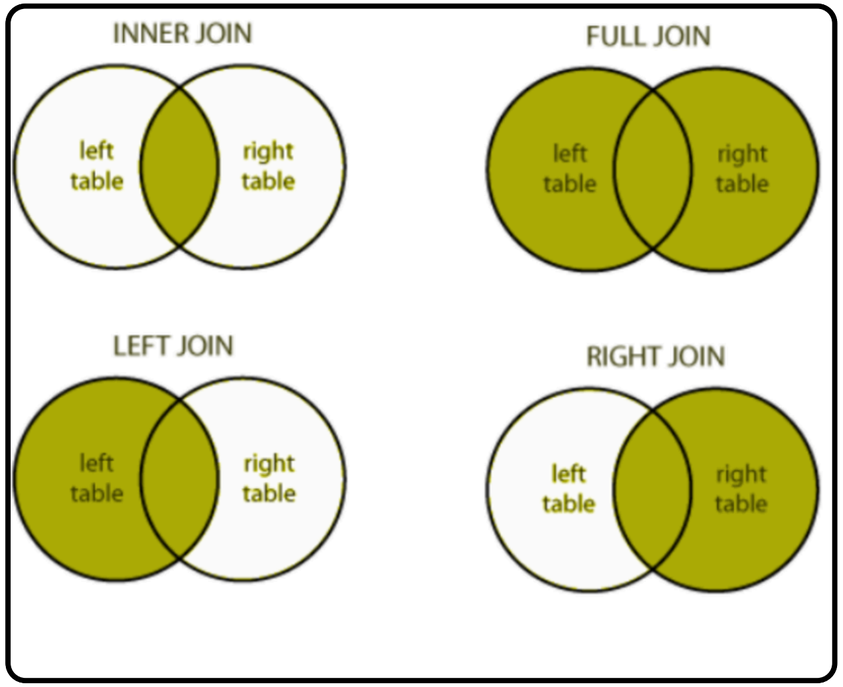
INNER e LEFT: duas formas de juntar

```
SELECT c.nome, t.valor
FROM clientes c
LEFT JOIN transacoes t
    ON c.id_cliente = t.id_cliente
```

INNER JOIN só traz quem existe nas duas tabelas: cliente sem nenhuma transação some do resultado

LEFT JOIN mantém tudo da tabela da esquerda: clientes sem transação aparecem, com valor vazio

A escolha muda a resposta: 'quantos clientes nunca compraram' só aparece com LEFT JOIN



INTERAÇÃO

Qual JOIN resolve isso

Qual consulta preserva quem não tem correspondência?

- A tabela lojistas possui 500 lojistas cadastrados.
- A tabela transacoes contém compras realizadas em apenas 430 deles.
- Você precisa descobrir quais 70 lojistas nunca receberam nenhuma transação.
- Qual estratégia permite que esses lojistas continuem aparecendo no resultado?

A) Começar por transacoes e usar INNER JOIN com lojistas
B) Começar por lojistas e usar LEFT JOIN com transacoes
C) Começar por lojistas e usar INNER JOIN com transacoes

INTERAÇÃO

RESPOSTA

Qual consulta preserva quem não tem correspondência?

- A tabela lojistas possui 500 lojistas cadastrados.
- A tabela transacoes contém compras realizadas em apenas 430 deles.
- Você precisa descobrir quais 70 lojistas nunca receberam nenhuma transação.
- Qual estratégia permite que esses lojistas continuem aparecendo no resultado?

```
SELECT lojistas.nome
FROM lojistas
LEFT JOIN transacoes
        ON lojistas.id_lojista = transacoes.id_lojista
WHERE transacoes.id_transacao IS NULL
```

White Cube
A Think Group Company

06

Da pergunta de negócio a query SQL

Fechando o dia e olhando para a quinta-feira

21h45 às 22h05 · 20 minutos

ENCERRAMENTO

Cinco ideias para levar de hoje:

Se você conseguir levar esses cinco conceitos, a noite cumpriu o objetivo.

1. **SQL descreve o que você quer, não o passo a passo:** quem decide o caminho é o banco
2. **A ordem de execução não é a ordem que você escreve:** FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT
3. **Agregação muda o formato da resposta:** de uma lista de linhas para um resumo
4. **JOIN existe porque os dados vivem separados de propósito:** e o tipo de JOIN muda o resultado
5. **Toda consulta boa começa numa pergunta de negócio clara:** não numa tabela aberta sem destino

CRONOGRAMA - AULAS SEGUINTES

SQL para Análise de Dados → Python para Análise de Dados



QUINTA-FEIRA · Semana 3

- Primeiro contato com SQL de verdade, num banco real
- Consultas na prática, com o caso de fraude
- Mesmo formato: demonstração e mão na massa
- Vale revisar os slides de hoje antes



SEGUNDA-FEIRA · Semana 4

- Análise de dados com Python
- Mesma lógica de hoje, agora com pandas
- Ainda conceitual, como toda segunda
- Ganchos diretos com o que vimos aqui

White Cube
A Think Group Company

Perguntas?

Escreva no chat. O que não der tempo, respondemos no Discord assim que possível.

RECAP: PROJETO FINAL

Qual será o projeto final que será desenvolvido ao longo do curso?

Teremos 4 rotas principais que poderão ser desenvolvidas.

Analytics

- Foco em explorar os dados e contar uma história a partir deles.
- Trabalha com análise exploratória, indicadores e visualização.

- Exemplo de entrega: um dashboard com os principais indicadores do problema escolhido, acompanhado da geração de um relatório executivo e automatizável com as descobertas

Engenharia de Dados

- Foco em construir a estrutura por trás da solução.
- Trabalha com pipelines de dados, automações e organização de código.

- Exemplo de entrega: um pipeline de dados e2e automatizado, ou uma API que consulta dados em uma fonte, transforma e entrega a informação pronta para uso.

Machine Learning

- Foco em prever ou classificar algo a partir de padrões nos dados.
- Trabalha com treino, teste, métricas e comparação de modelos.

- Exemplo de entrega: um modelo preditivo, de classificação ou de agrupamento, com toda a metodologia de desenvolvimento e métricas que comprovam a qualidade

GenAI e Agentes

- Foco em usar modelos de linguagem para resolver partes do problema ou todo ele.
- Trabalha com prompt, contexto e avaliação de resposta.

- Exemplo de entrega: um copiloto ou assistente com capacidades de RAG, automatizando toda ou parte de uma tarefa com LLM e uso de tools (ferramentas).

White Cube
A Think Group Company

Obrigado!

Nos vemos na quinta-feira, 19h00.

Discord do AI Talent Academy: link fixado no chat e enviado por e-mail.
Gravação e slides desta aula chegam em até 24h.